

INSTITUTO FEDERAL SUL RIO-GRANDENSE
CÂMPUS JAGUARÃO
CURSO TÉCNICO INTEGRADO EM INFORMÁTICA PARA INTERNET

Pedro Joaquim Rocha Junes

SGD
Software Gerenciador de Dívidas

Jaguarão
2025

INSTITUTO FEDERAL SUL RIO-GRANDENSE
CÂMPUS JAGUARÃO
CURSO TÉCNICO INTEGRADO EM INFORMÁTICA PARA INTERNET

Pedro Joaquim Rocha Junes

SGD
Software Gerenciador de Dívidas

Trabalho de Conclusão de Curso
apresentando o desenvolvimento de
uma aplicação Java visando facilitar a
gestão de um mercado de pequeno
porte.

Jaguarão
2025

INSTITUTO FEDERAL SUL RIO-GRANDENSE
CÂMPUS JAGUARÃO
CURSO TÉCNICO INTEGRADO EM INFORMÁTICA PARA INTERNET

Pedro Joaquim Rocha Junes

SGD

Software Gerenciador de Dívidas

Trabalho de Conclusão de Curso
apresentado no Instituto Federal Sul
Rio-grandense como requisito parcial
para obtenção do certificado de
conclusão do Curso Técnico Integrado
em Informática para Internet.

BANCA EXAMINADORA

Orientador: Timóteo Matthies Rico
Instituto Federal Sul Rio-grandense

Professora: Evelyn Oliveira Inacio
Instituto Federal Sul Rio-grandense

Professor: Marcos Ceron Gonçalves
Instituto Federal Sul Rio-grandense

Professor: Vinícius Alves Hax
Instituto Federal Sul Rio-grandense

SUMARIO

SUMARIO.....	4
RESUMO.....	5
ABSTRACT.....	5
1 INTRODUÇÃO.....	5
1.1 JUSTIFICATIVA.....	5
1.2 OBJETIVOS GERAIS.....	6
1.3 OBJETIVOS ESPECÍFICOS.....	6
2 METODOLOGIA.....	6
2.1 BANCO DE DADOS.....	7
2.1.2 Modelo ER.....	7
2.1.2 Tabelas.....	7
2.1.2.1 Tabela buyer.....	7
2.1.2.2 Tabela bill.....	7
2.1.2.3 Tabela products.....	7
2.1.2.4 Tabla brand.....	8
2.1.3 Interações Java - PostgreSQL.....	8
2.2 VALIDAÇÕES.....	9
2.2.1 Validando produtos.....	9
2.2.2 Validando clientes.....	10
2.2.3 Validando vendas.....	11
2.2.4 Validando CPF.....	12
3 RESULTADOS E DISCUSSÕES.....	13
3.1 TELAS.....	13
3.1.1 Tela de Login.....	13
3.1.2 Tela de Menu.....	14
3.1.3 Tela de Cadastrar Clientes.....	15
3.1.4 Tela de Atualizar Clientes.....	16
3.1.5 Tela de Deletar Clientes.....	17
3.1.6 Tela de Pesquisar Clientes.....	18
3.1.7 Tela de Cadastrar Produto.....	19
3.1.8 Tela de Atualizar Produto.....	20
3.1.9 Tela de Deletar Produto.....	21
3.1.10 Tela de Pesquisar Produtos.....	22
3.1.11 Tela de Editar Marcas.....	23
3.1.12 Tela de Cobrança.....	24
4 CONCLUSÕES.....	25
REFERENCIAS BIBLIOGRÁFICAS.....	25

RESUMO

Visando que a maior parte dos pequenos mercados na cidade Jaguarão não possuem um sistema que auxilie no gerenciamento de vendas e clientes, este projeto foi desenvolvido com a intenção de preencher este espaço no mercado. O projeto foi nomeado de SGD (Software Gerenciador de Dívidas). O software foi desenvolvido na linguagem de programação Java, utilizando o banco de dados PostgreSQL. Tendo como público alvo o pequeno empreendedor, a interface é simples e intuitiva, oferecendo praticidade e agilidade ao operador.

Palavras-chave: Software; Java; PostgreSQL

ABSTRACT

Considering that most small markets in the city of Jaguarão do not have a system that helps with sales and customer management, this project was developed with the intention of filling this space in the market. The project was named DMS (Debt Management Software). The software was developed in the programming language Java, using the PostgreSQL database. Targeting small entrepreneurs, the interface is simple and intuitive, offering practicality and agility to the operator.

Keywords: Software; Java; PostgreSQL

1 INTRODUÇÃO

1.1 JUSTIFICATIVA

Este trabalho foi pensado como uma proposta de auxiliar os mercados de pequeno porte. Visto que grande quantidade desses estabelecimentos não possui nenhum tipo de sistema de controle. Usar um software de gerenciamento ao invés de um caderno traz diversas vantagens, principalmente em termos de organização, eficiência e segurança, por exemplo em um software de gerenciamento você pode buscar informações rapidamente por meio de filtros e pesquisas, no entanto usando um caderno, isto não é possível.

1.2 OBJETIVOS GERAIS

Desenvolver um software desktop para a gestão de um mercado de pequeno porte.

1.3 OBJETIVOS ESPECÍFICOS

- Gerenciar produtos
- Gerenciar clientes
- Gerenciar vendas

2 METODOLOGIA

Para o desenvolvimento do sistema foi utilizado o ambiente de desenvolvimento integrado NetBeans e a linguagem de programação Java integrada ao banco de dados PostgreSQL.

NetBeans é um ambiente de desenvolvimento integrado para Java que fornece ferramentas e utilidades que facilitam o desenvolvimento de softwares em Java.

Java é uma linguagem de programação de alto nível, baseada em classes e programação orientada a objetos, a linguagem permite que programadores escrevam uma vez e executem em qualquer lugar. O que significa que o código Java compilado pode ser executado em todas as plataformas que suportem Java sem necessidade de recompilação.

PostgreSQL é um sistema de gerenciamento de banco de dados, o sistema é gratuito e é código-aberto.

Estas duas ferramentas foram escolhidas para a produção deste software pois oferecem recursos essenciais para seu funcionamento.

2.1 BANCO DE DADOS

2.1.2 Modelo ER

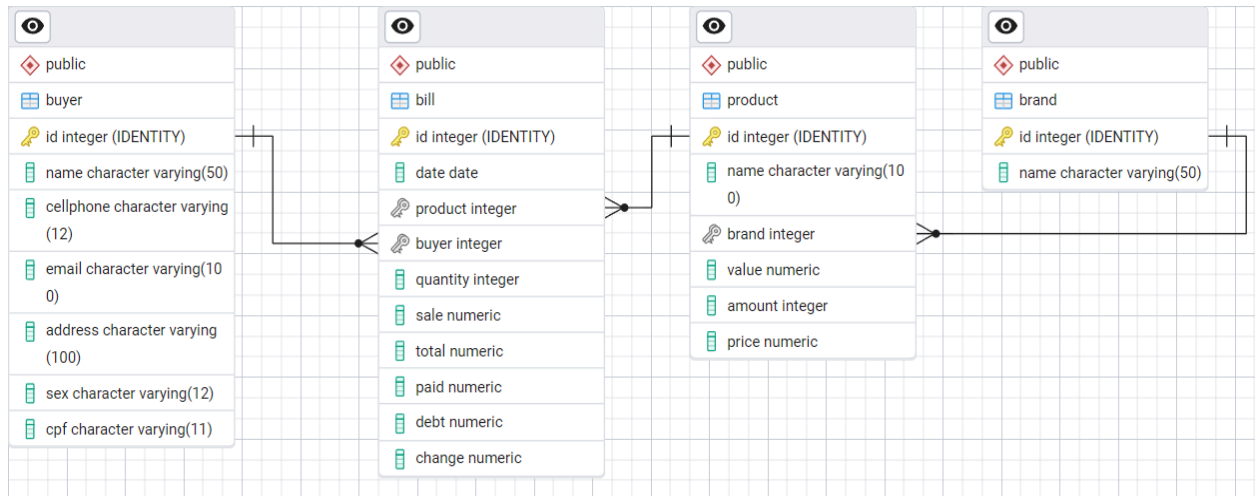


Figura 2.1 - Modelo ER ; Fonte : Arquivo pessoal.

2.1.2 Tabelas

2.1.2.1 Tabela buyer

Possui os campos “name”, “cellphone”, “email”, “address”, “sex”, “cpf”. Como sugeridos pelos nomes, estes campos armazenam respectivamente o nome, celular, email, endereço, sexo e CPF do cliente.

2.1.2.2 Tabela bill

Possui os campos “date”, “product”, “buyer”, “quantity”, “sale”, “total”, “paid”, “debt” e “change”, estes campos armazenam respectivamente a data que uma venda foi efetuada, o id referente ao produto, o id referente ao cliente, valor unitário da venda do produto, valor total da venda, valor pago pelo cliente, o quanto o cliente ficou devendo e o troco.

2.1.2.3 Tabela products

Possui os campos “name”, “brand”, “value”, “amount” e “price”. Estes campos armazenam respectivamente o nome do produto, id referente a marca, valor de mercado, quantidade disponível para venda, e o preço do produto.

2.1.2.4 Tabla brand

Possui o campo “name”, este campo armazena o nome da marca.

2.1.3 Interações Java - PostgreSQL

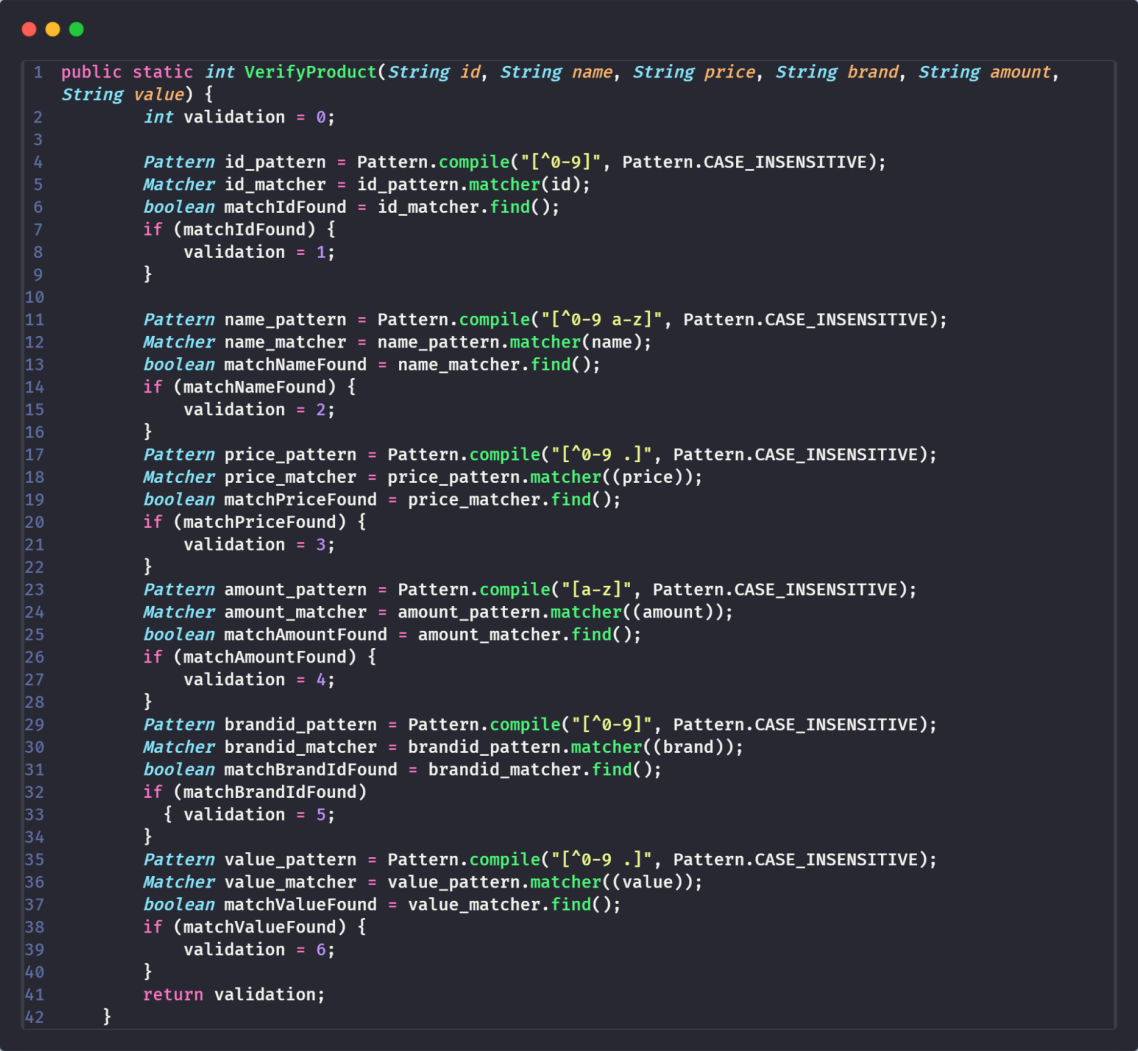


Figura 2.2 - Conexão com banco de dados ; Fonte : Arquivo pessoal.

Classe “ConnectionProvider”, responsável pela conexão do Java com o banco de dados PostgreSQL.

2.2 VALIDAÇÕES

2.2.1 Validando produtos

A screenshot of a code editor with a dark background and light-colored text. The code is in Java and defines a static method named `VerifyProduct`. The method takes five string parameters: `id`, `name`, `price`, `brand`, and `amount`, and returns an integer `validation`. The code uses regular expressions to validate each parameter. It starts with `validation = 0` and increments it for each successful validation. The validations are: `id` (digits), `name` (alphanumeric), `price` (alphanumeric with a decimal point), `amount` (alphanumeric), `brand` (alphanumeric), and `value` (alphanumeric with a decimal point).

```
1 public static int VerifyProduct(String id, String name, String price, String brand, String amount,
2 String value) {
3     int validation = 0;
4
5     Pattern id_pattern = Pattern.compile("[^0-9]", Pattern.CASE_INSENSITIVE);
6     Matcher id_matcher = id_pattern.matcher(id);
7     boolean matchIdFound = id_matcher.find();
8     if (matchIdFound) {
9         validation = 1;
10    }
11
12    Pattern name_pattern = Pattern.compile("[^0-9 a-z]", Pattern.CASE_INSENSITIVE);
13    Matcher name_matcher = name_pattern.matcher(name);
14    boolean matchNameFound = name_matcher.find();
15    if (matchNameFound) {
16        validation = 2;
17    }
18
19    Pattern price_pattern = Pattern.compile("[^0-9 .]", Pattern.CASE_INSENSITIVE);
20    Matcher price_matcher = price_pattern.matcher((price));
21    boolean matchPriceFound = price_matcher.find();
22    if (matchPriceFound) {
23        validation = 3;
24    }
25
26    Pattern amount_pattern = Pattern.compile("[a-z]", Pattern.CASE_INSENSITIVE);
27    Matcher amount_matcher = amount_pattern.matcher((amount));
28    boolean matchAmountFound = amount_matcher.find();
29    if (matchAmountFound) {
30        validation = 4;
31    }
32
33    Pattern brandid_pattern = Pattern.compile("[^0-9]", Pattern.CASE_INSENSITIVE);
34    Matcher brandid_matcher = brandid_pattern.matcher((brand));
35    boolean matchBrandIdFound = brandid_matcher.find();
36    if (matchBrandIdFound) {
37        validation = 5;
38    }
39
40    Pattern value_pattern = Pattern.compile("[^0-9 .]", Pattern.CASE_INSENSITIVE);
41    Matcher value_matcher = value_pattern.matcher((value));
42    boolean matchValueFound = value_matcher.find();
43    if (matchValueFound) {
44        validation = 6;
45    }
46
47    return validation;
48 }
```

Figura 2.3 - Validação de produtos ; Fonte : Arquivo pessoal.

Este método por meio de expressões regulares faz a validação dos dados inseridos pelo usuário.

2.2.2 Validando clientes

```
1 public static int VerifyBuyer(String id, String name, String cellphone, String email, String address,  
2 String CPF, String gender) {  
3     int validation = 0;  
4  
5     Pattern id_pattern = Pattern.compile("[0-9]", Pattern.CASE_INSENSITIVE);  
6     Matcher id_matcher = id_pattern.matcher(id);  
7     boolean matchIdFound = id_matcher.find();  
8     if (matchIdFound) {  
9         validation = 1;  
10    }  
11    Pattern name_pattern = Pattern.compile("[0-9]", Pattern.CASE_INSENSITIVE);  
12    Matcher name_matcher = name_pattern.matcher(name);  
13    boolean matchNameFound = name_matcher.find();  
14    if (matchNameFound) {  
15        validation = 2;  
16    }  
17    Pattern cellphone_pattern = Pattern.compile("^((\\d{2}\\d{2})?)? \\d{4,5}-\\d{4}$",  
18    , Pattern.CASE_INSENSITIVE);  
19    Matcher cellphone_matcher = cellphone_pattern.matcher(cellphone);  
20    boolean matchCellphoneFound = cellphone_matcher.find();  
21    if (matchCellphoneFound == false) {  
22        validation = 3;  
23    }  
24    Pattern email_pattern = Pattern.compile("^([\\x00-\\x20\\x22\\x28\\x29\\x2c\\x2e\\x3a-\\x3c\\x3e\\x40\\x5b-\\x5d\\x7f-\\xff]|\\x22([\\x0d\\x22\\x5c\\x80-\\xff]|\\x5c[\\x00-\\x7f])*\\x22)(\\x22([\\x00-\\x20\\x22\\x28\\x29\\x2c\\x2e\\x3a-\\x3c\\x3e\\x40\\x5b-\\x5d\\x7f-\\xff]|\\x22([\\x0d\\x22\\x5c\\x80-\\xff]|\\x5c[\\x00-\\x7f])*\\x22))*\\x40([\\x00-\\x20\\x22\\x28\\x29\\x2c\\x2e\\x3a-\\x3c\\x3e\\x40\\x5b-\\x5d\\x7f-\\xff]|\\x5b([\\x0d\\x5b-\\x5d\\x80-\\xff]|\\x5c[\\x00-\\x7f])*\\x5d)(\\x2e([\\x00-\\x20\\x22\\x28\\x29\\x2c\\x2e\\x3a-\\x3c\\x3e\\x40\\x5b-\\x5d\\x7f-\\xff]|\\x5b([\\x0d\\x5b-\\x5d\\x80-\\xff]|\\x5c[\\x00-\\x7f])*\\x5d))*$", Pattern.CASE_INSENSITIVE);  
25    Matcher email_matcher = email_pattern.matcher(email);  
26    boolean matchEmailFound = email_matcher.find();  
27    if (matchEmailFound == false) {  
28        validation = 4;  
29    }  
30    Pattern address_pattern = Pattern.compile("[^ A-Z 0-9 ç]", Pattern.CASE_INSENSITIVE);  
31    Matcher address_matcher = address_pattern.matcher(address);  
32    boolean matchAddressFound = address_matcher.find();  
33    if (matchAddressFound) {  
34        validation = 5;  
35    }  
36    Pattern gender_pattern = Pattern.compile("^1|2", Pattern.CASE_INSENSITIVE);  
37    Matcher gender_matcher = gender_pattern.matcher(gender);  
38    boolean matchGenderFound = gender_matcher.find();  
39    if (matchGenderFound) {  
40        validation = 6;  
41    }  
42    return validation;  
43 }
```

Figura 2.4 - Validação de clientes ; Fonte : Arquivo pessoal.

Este método por meio de expressões regulares faz a validação dos dados inseridos pelo usuário.

2.2.3 Validando vendas

```
1 public static int VerifyBill(String id,String product_id, String buyer_id, String quantity, String sale
2 ){
3     int validation = 0;
4
5     Pattern id_pattern = Pattern.compile("[^0-9]", Pattern.CASE_INSENSITIVE);
6     Matcher id_matcher = id_pattern.matcher(id);
7     boolean matchIdFound = id_matcher.find();
8     if (matchIdFound) {
9         validation = 1;
10    }
11    Pattern product_id_pattern = Pattern.compile("[^ 0-9]", Pattern.CASE_INSENSITIVE);
12    Matcher product_id_matcher = product_id_pattern.matcher(product_id);
13    boolean matchProductIdFound = product_id_matcher.find();
14    if (matchProductIdFound) {
15        validation = 2;
16    }
17    Pattern buyer_id_pattern = Pattern.compile("[^ 0-9]", Pattern.CASE_INSENSITIVE);
18    Matcher buyer_id_matcher = buyer_id_pattern.matcher(buyer_id);
19    boolean matchBuyerIdFound = buyer_id_matcher.find();
20    if (matchBuyerIdFound)
21    { validation = 3;
22    }
23    Pattern quantity_pattern = Pattern.compile("[^ 0-9]", Pattern.CASE_INSENSITIVE);
24    Matcher quantity_matcher = quantity_pattern.matcher(quantity);
25    boolean matchQuantityIdFound = quantity_matcher.find();
26    if (matchQuantityIdFound) {
27        validation = 5;
28    }
29    Pattern sale_pattern = Pattern.compile("[^ 0-9 .]", Pattern.CASE_INSENSITIVE);
30    Matcher sale_matcher = sale_pattern.matcher(sale);
31    boolean matchSaleIdFound = sale_matcher.find();
32    if (matchSaleIdFound) {
33        validation = 7;
34    }
35    return validation;
36 }
```

Figura 2.5 - Validação de vendas ; Fonte : Arquivo pessoal.

Este método por meio de expressões regulares faz a validação dos dados inseridos pelo usuário.

2.2.4 Validando CPF

```
1 public static boolean isCPF(String CPF) {
2     if (CPF.equals("00000000000"))
3         || CPF.equals("11111111111")
4         || CPF.equals("22222222222") || CPF.equals("33333333333")
5         || CPF.equals("44444444444") || CPF.equals("55555555555")
6         || CPF.equals("66666666666") || CPF.equals("77777777777")
7         || CPF.equals("88888888888") || CPF.equals("99999999999")
8         || (CPF.length() != 11)) {
9         return false;
10    }
11    char dig10, dig11;
12    int sm, i, r, num, peso;
13    try {
14        sm = 0;
15        peso = 10;
16        for (i = 0; i < 9; i++) {
17            num = (int) (CPF.charAt(i) - 48);
18            sm = sm + (num * peso);
19            peso = peso - 1;
20        }
21        r = 11 - (sm % 11);
22        if ((r == 10) || (r == 11)) {
23            dig10 = '0';
24        } else {
25            dig10 = (char) (r + 48);
26        }
27        sm = 0;
28        peso = 11;
29        for (i = 0; i < 10; i++) {
30            num = (int) (CPF.charAt(i) - 48);
31            sm = sm + (num * peso);
32            peso = peso - 1;
33        }
34        r = 11 - (sm % 11);
35        if ((r == 10) || (r == 11)) {
36            dig11 = '0';
37        } else {
38            dig11 = (char) (r + 48);
39        }
40        if ((dig10 == CPF.charAt(9)) && (dig11 == CPF.charAt(10))) {
41            return true;
42        } else {
43            return false;
44        }
45    } catch (InputMismatchException erro) {
46        return false;
47    }
48 }
```

Figura 2.6 - Validação de CPF ; Fonte : Arquivo pessoal.

Este método utiliza o cálculo do dígito verificador do cpf para validar o cpf inserido pelo usuário.

3 RESULTADOS E DISCUSSÕES

3.1 TELAS

3.1.1 Tela de Login



Figura 3.1 - Tela Login ; Fonte: Arquivo pessoal.

Nesta tela o usuário efetuará o login, para que possa acessar as opções administrativas.

3.1.2 Tela de Menu



Figura 3.2 - Tela de menu ; Fonte : Arquivo pessoal.

Caso o usuário tenha feito login, essa tela será iniciada. Nela estão todas as opções disponíveis no sistema.

3.1.3 Tela de Cadastrar Clientes



Adicionar cliente

File

 **ADICIONAR CLIENTE**

Nome

Telefone

Email

Endereço

CPF

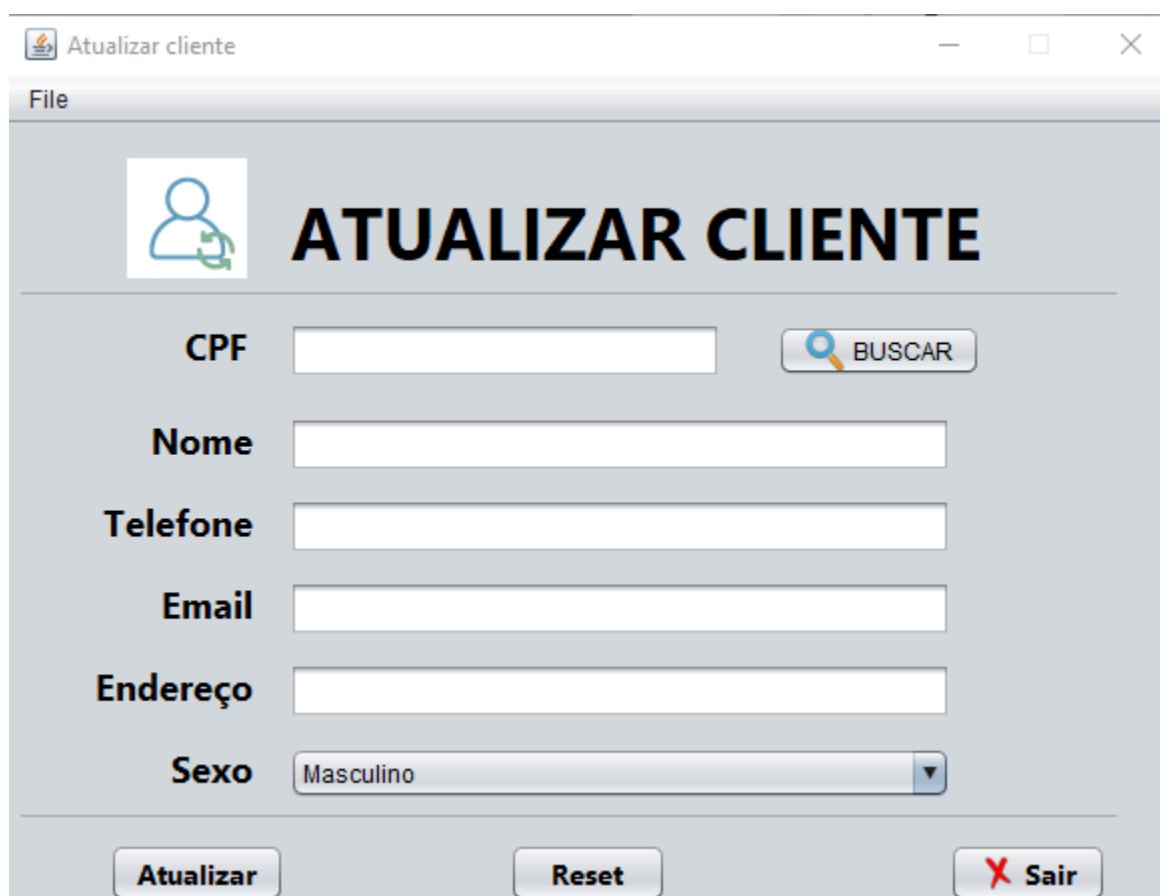
Sexo

 **Salvar**  **Resetar**  **Sair**

Figura 3.3 - Tela de cadastrar clientes ; Fonte: Arquivo pessoal.

Esta é a tela onde o usuário poderá realizar o cadastro de novos clientes.

3.1.4 Tela de Atualizar Clientes



Atualizar cliente

File

 **ATUALIZAR CLIENTE**

CPF  **BUSCAR**

Nome

Telefone

Email

Endereço

Sexo

Atualizar **Reset** **X Sair**

Figura 3.4 - Tela de Atualizar clientes ; Fonte: Arquivo pessoal.

Nesta tela os clientes cadastrados no sistema podem ter seus dados atualizados.

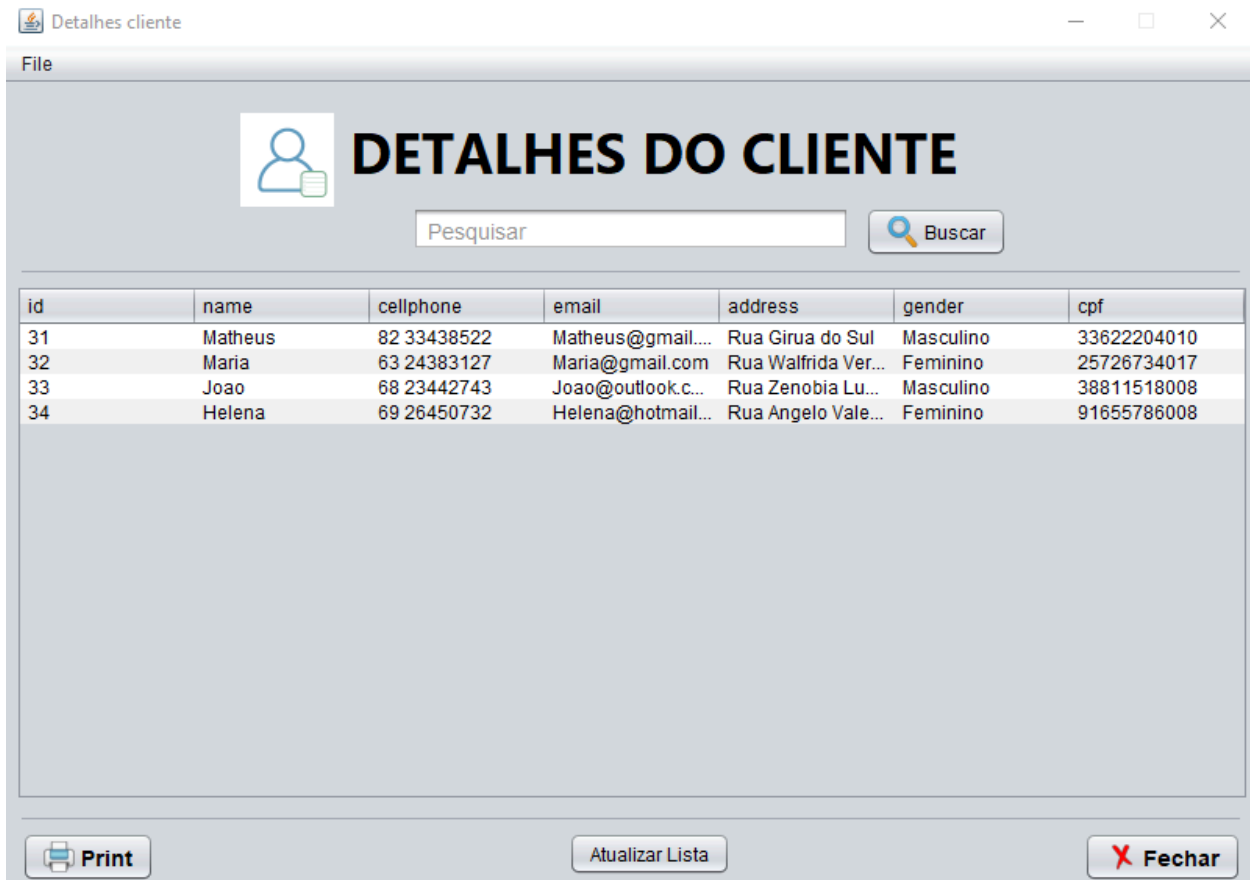
3.1.5 Tela de Deletar Clientes

The screenshot shows a web application window titled "Deletar cliente". The interface has a light gray background. At the top left, there is a menu bar with the word "File". Below the menu bar, on the left, is a user icon (a blue circle with a white person silhouette) and a red circle with a white "X" over it. To the right of this icon, the text "DELETAR CLIENTE" is displayed in large, bold, black capital letters. Below this header section, there is a search bar labeled "CPF" with a text input field and a "BUSCAR" button with a magnifying glass icon. Underneath the search bar, there are five more input fields, each with a label to its left: "Nome", "Telefone", "Email", "Endereço", and "Sexo". At the bottom of the interface, there are three buttons: "Deletar" with a red trash can icon, "Resetar" with a blue circular arrow icon, and "Sair" with a red "X" icon.

Figura 3.5 - Tela de Deletar Clientes; Fonte : Arquivo pessoal.

Nesta tela o usuário poderá deletar os clientes cadastrados no sistema.

3.1.6 Tela de Pesquisar Clientes



The screenshot shows a window titled 'Detalhes cliente' with a standard Windows-style title bar (minimize, maximize, close buttons). The window has a menu bar with 'File'. The main content area has a header section with a user icon, the title 'DETALHES DO CLIENTE', a search input field labeled 'Pesquisar', and a 'Buscar' button. Below this is a table with 7 columns: id, name, cellphone, email, address, gender, and cpf. The table contains 4 rows of client data. At the bottom of the window are three buttons: 'Print', 'Atualizar Lista', and 'Fechar' (with a red X icon).

id	name	cellphone	email	address	gender	cpf
31	Matheus	82 33438522	Matheus@gmail....	Rua Girua do Sul	Masculino	33622204010
32	Maria	63 24383127	Maria@gmail.com	Rua Walfrida Ver...	Feminino	25726734017
33	Joao	68 23442743	Joao@outlook.c...	Rua Zenobia Lu...	Masculino	38811518008
34	Helena	69 26450732	Helena@hotmail...	Rua Angelo Vale...	Feminino	91655786008

Figura 3.6 - Tela de Pesquisar clientes ; Fonte : Arquivo pessoal.

Nesta tela o usuário poderá pesquisar pelos dados dos clientes cadastrados no sistema.

3.1.7 Tela de Cadastrar Produto

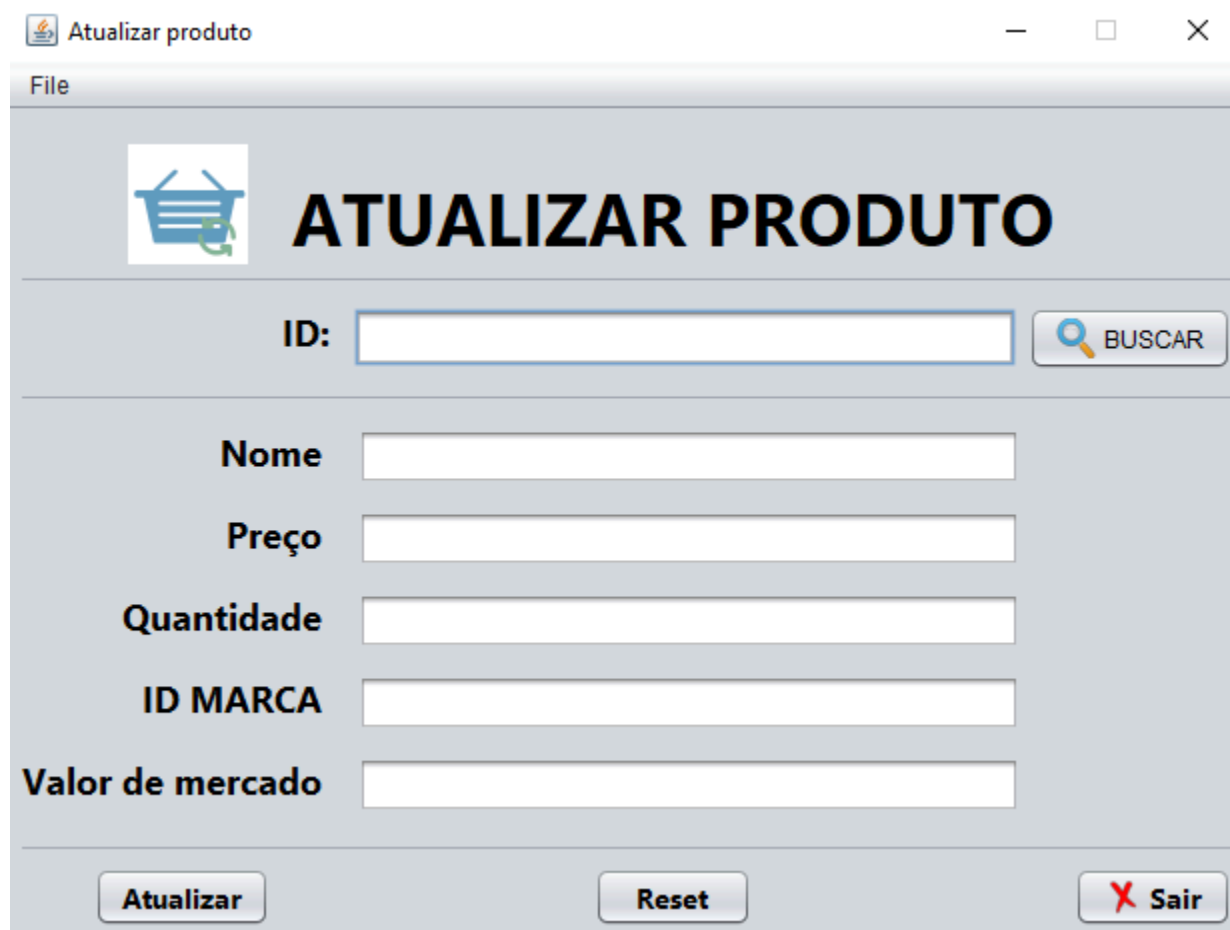


The image shows a software window titled "Adicionar produto" with standard Windows window controls (minimize, maximize, close). The window has a menu bar with "File". The main content area has a header with a shopping cart icon and the text "NOVO PRODUTO". Below this, there are five labeled input fields: "Nome", "Preço", "Quantidade", "ID Marca", and "Valor de mercado". At the bottom of the window, there are three buttons: "Salvar" (with a floppy disk icon), "Resetar" (with a circular arrow icon), and "Fechar" (with a red X icon).

Figura 3.7 - Tela de Cadastrar Produto; Fonte: Arquivo pessoal.

Esta é a tela onde o usuário poderá adicionar novos produtos ao sistema.

3.1.8 Tela de Atualizar Produto



File

 **ATUALIZAR PRODUTO**

ID:  **BUSCAR**

Nome

Preço

Quantidade

ID MARCA

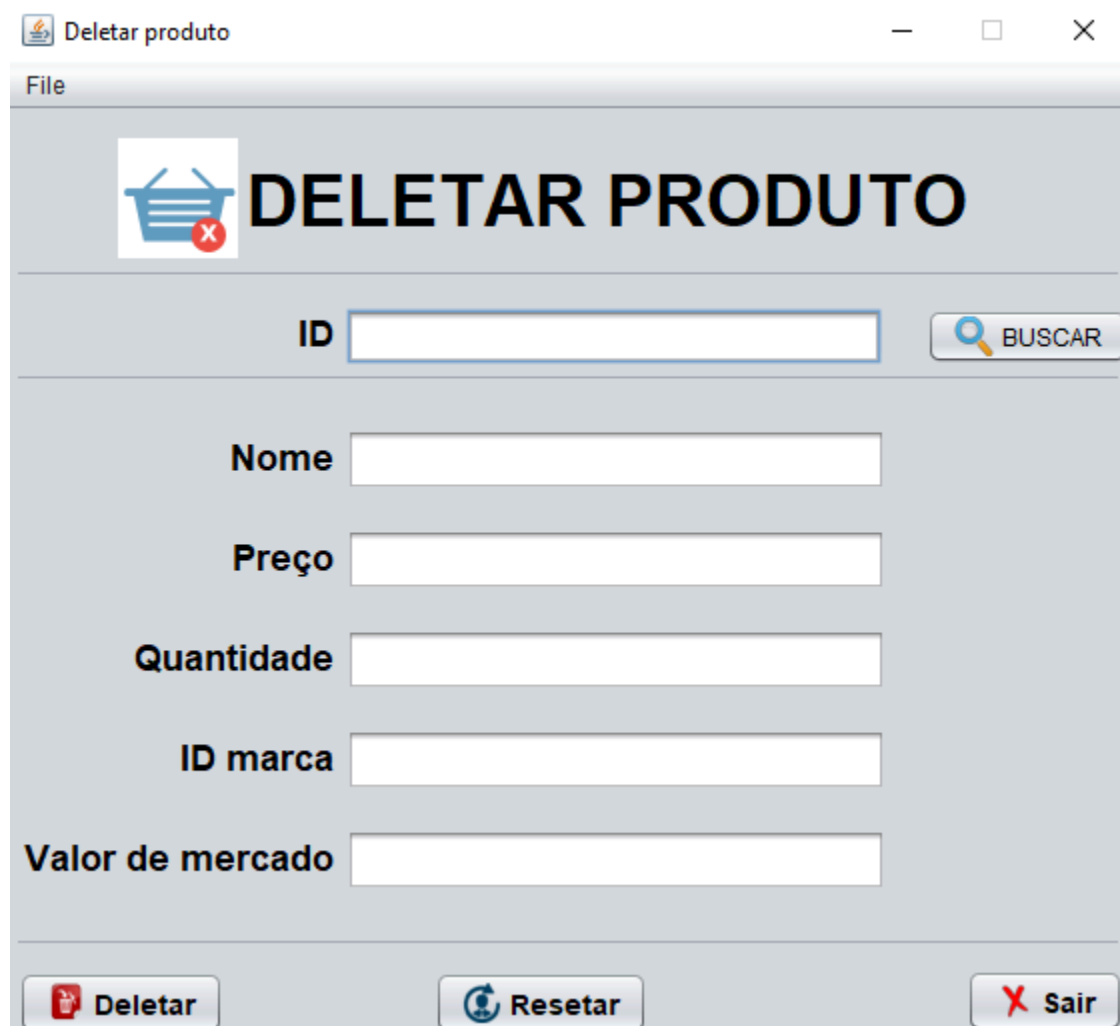
Valor de mercado

Atualizar **Reset**  **Sair**

Figura 3.8 - Tela de Atualizar Produto ; Fonte: Arquivo pessoal.

Nesta tela o usuário pode atualizar os dados dos produtos cadastrados no sistema.

3.1.9 Tela de Deletar Produto



Deletar produto

File

 **DELETAR PRODUTO**

ID  **BUSCAR**

Nome

Preço

Quantidade

ID marca

Valor de mercado

 **Deletar**  **Resetar**  **Sair**

Figura 3.1.9 - Tela Deletar produto; Fonte : Arquivo pessoal.

Nesta tela o usuário poderá deletar os produtos cadastrados no sistema.

3.1.10 Tela de Pesquisar Produtos

Detalhes do produto

File

DETALHES DO PRODUTO

id	name	brand	value	amount	price
4	Frango	15	10.5	10	12.5
15	Picanha	1	20	10	50
1	bolacha doce	1	2	10	5
19	bolacha verde	1	2.5	10	5.5
20	Pizza	15	10	5	20
18	bolacha rosa	1	1	1	1
25	Coca cola 2L	31	5	5	10.5
3	Agua Mineral 500ml	1	1	10	2.5

Figura 3.10 - Tela de Pesquisar produtos; Fonte: Arquivo pessoal.

O usuário pode pesquisar pelos dados dos produtos cadastrados nesta tela.

3.1.11 Tela de Editar Marcas

Editor de marcas

EDITOR DE MARCAS

NOME OU ID:

NOME:

ID:

id	name
1	zeze
2	Cocacola
15	Sadia
16	Vale
26	Nestle
31	CocaCola

Figura 3.11 - Tela Editor de marcas; Fonte: Arquivo pessoal.

Esta tela é utilizada para cadastrar marcas, pesquisá-las e deletá-las.

3.1.12 Tela de Cobrança

Editor de contas

File

Adicionar Editar

GERAR CONTA Data: 2025-02-09

DETALHES DA VENDA

ID CLIENTE

ID PRODUTO:

QUANTIDADE:

VALOR DE VENDA:

VALOR PAGO:

CALCULOS

Total

Troco

Calcular

CONTAS

id	date	pro...	buyer	qua...	sale	total	paid	debt	cha...
----	------	--------	-------	--------	------	-------	------	------	--------

Pesquisar

Salvar Reiniciar Sair

Figura 3.12 - Tela de cobrança; Fonte : Arquivo pessoal.

Editor de contas

File

Adicionar Editar

EDITAR VENDAS Data: 2025-02-14

DETALHES DA VENDA

ID VENDA:

ID CLIENTE:

ID PRODUTO:

QUANTIDADE:

VALOR DE VENDA:

VALOR PAGO:

CONTAS

id	date	product	buyer	quantity	sale	total	paid	debt	change
----	------	---------	-------	----------	------	-------	------	------	--------

Pesquisar

Salvar Print Atualizar lista Deletar Sair

Figura 3.12.1 - Tela de cobrança; Fonte : Arquivo pessoal.

Nestas telas o usuário poderá cadastrar suas vendas e analisar o quanto seus clientes lhe devem, é possível pesquisar, atualizar e apagar as vendas.

4 CONCLUSÕES

O trabalho apresentou uma proposta de software para auxiliar no gerenciamento de um estabelecimento comercial de pequeno porte, com foco nas possíveis necessidades de um mercado. Apresenta recursos simples, porém com grande utilidade, principalmente se comparado ao caderno, que é o sistema utilizado pelos estabelecimentos em que essa aplicação é focada.

Novas funções podem ser adicionadas em implementações futuras, tais como mais opções de listagem, relatórios e mudanças na parte visual, podendo tornar a interface mais intuitiva e prática.

REFERENCIAS BIBLIOGRÁFICAS

Fireship. Regular Expressions (RegEx) in 100 Seconds. Disponível em: <<https://www.youtube.com/watch?v=sXQxhojSdZM>>. Acesso em : 09 de fevereiro de 2025

Regex para validar emails. **Regex 101**, 2025. Disponível em: <<https://regex101.com/r/sl6yF5/1>>. Acesso em: 09 de fevereiro de 2025.

Omero. Validando o CPF em uma aplicação Java. **DevMedia** , 2011. Disponível em: <<https://www.devmedia.com.br/validando-o-cpf-em-uma-aplicacao-java/22097>>. Acesso em : 09 de fevereiro de 2025.